

PDFs, document context, and citations

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 9 of 12 · English notes

PDFs join the same Messages array as another content type: a **document** block with `media_type: application/pdf`, usually base64 of the file bytes (class also mentioned reading via PyPDF when you extract text yourself). You may send one or many documents. The hard part is **context-window arithmetic** and telling Claude to trust *only* those files — then turning on **citations** so the UI can show where a claim came from.

Document blocks versus DIY text extraction

Two legitimate patterns: (1) native PDF support — send the document so Claude’s file pipeline parses pages (text + layout/vision as the model allows); (2) extract text locally and send a text block. Native documents keep page structure for citations. Either way, measure tokens before you fire. Classroom windows were ~272K vs 1M; pick the model whose input window fits the pack. Token-count APIs / metadata help you fail fast.

Prompt: “work only on the documents you see”

The model already “knows” machine learning from pretraining. If the user uploaded a specific ML PDF, say so: answer from **this** knowledge, not general memory. Then ask for mistakes, a topic summary, etc. That instruction is how you reduce fluent but ungrounded answers.

Enabling citations

On the document (and in current APIs, related citation flags), set citations enabled and optionally a **title** so multiple files are distinguishable. The assistant text blocks then carry a `citations` array you can render as footnotes or highlights. Vijay’s machine was struggling that day; he walked the fields more than he ran them — still examinable.

Source you grounded on	Class name	What it points at
A PDF / document block	citation document location	Where in that file (page / span as the API returns)
Plain text you put in messages	citation char / chat location (spoken “share/char”)	Character offsets in that text block

If the data source is a document, expect document locations. If you only stuffed a text blob into the prompt and enabled citations on it, expect character locations. Do not mix the names on the exam.

Why citations are not RAG by themselves

You still uploaded (or extracted) the whole file into *this* request’s context. RAG would retrieve chunks from a store first. Citations tell you which span supported a sentence *inside the context you already sent*. Cache (next chapter) is how you stop paying to re-send a fat PDF every turn.

Code execution aside (same session, not this chapter's exam core)

Vijay listed **code execution** as another advanced feature: the model writes a Python script to build a PDF rather than emitting a binary PDF. That is a tool/sandbox story (later modules). Do not confuse “Claude created a PDF” with “we sent a PDF as context.”

SIZE BEFORE SEND

A 1M-token PDF on every user click is an input bill, not a flex. Check token estimates, split files, or cache the document prefix.

UI use

Citation payloads exist so your application can jump to page and quote. Dumping raw citation JSON on a business user is not the feature; highlighting is.

DOCS OVER THE HUNG VS CODE DEMO

Field names for citation types evolve. Use **citations** and **PDF support** as the key.

Official documentation

- **PDF support**

<https://docs.claude.com/en/docs/build-with-claude/pdf-support>

Document blocks, page limits, combining with prompts.

- **Citations**

<https://docs.claude.com/en/docs/build-with-claude/citations>

Enabling citations and location types.

- **Vision**

<https://docs.claude.com/en/docs/build-with-claude/vision>

When a page is better sent as images.

- **Messages API**

<https://docs.claude.com/en/api/messages>

Content block types including document.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.